

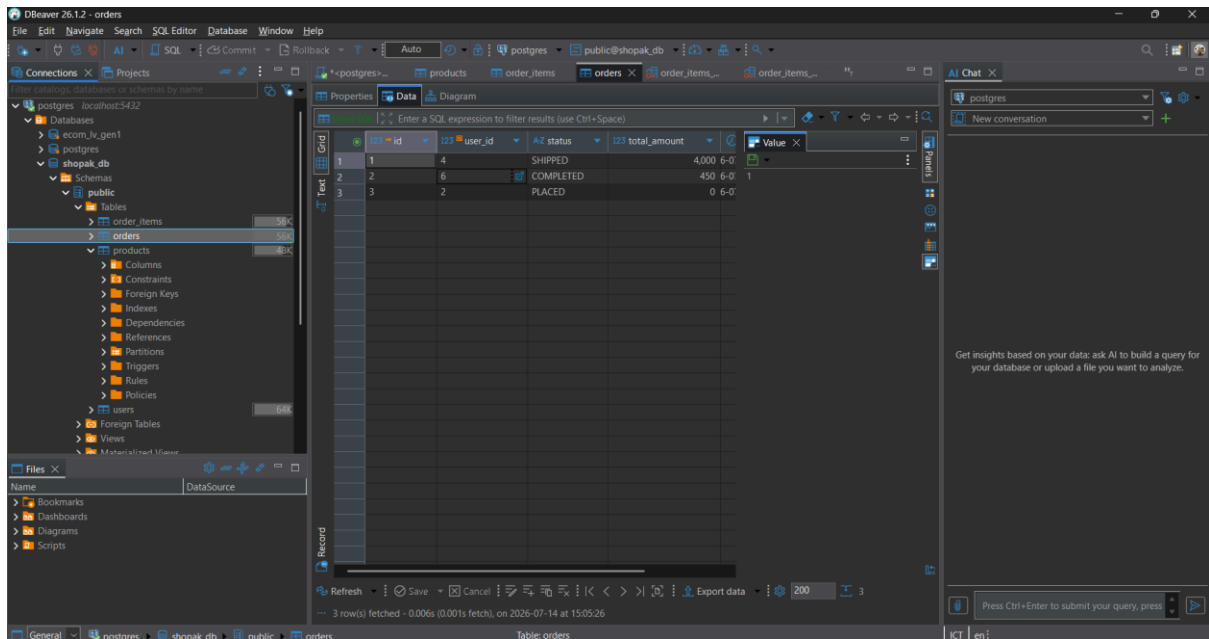
Session12_10.07.2026

Lab Tasks

1. Database

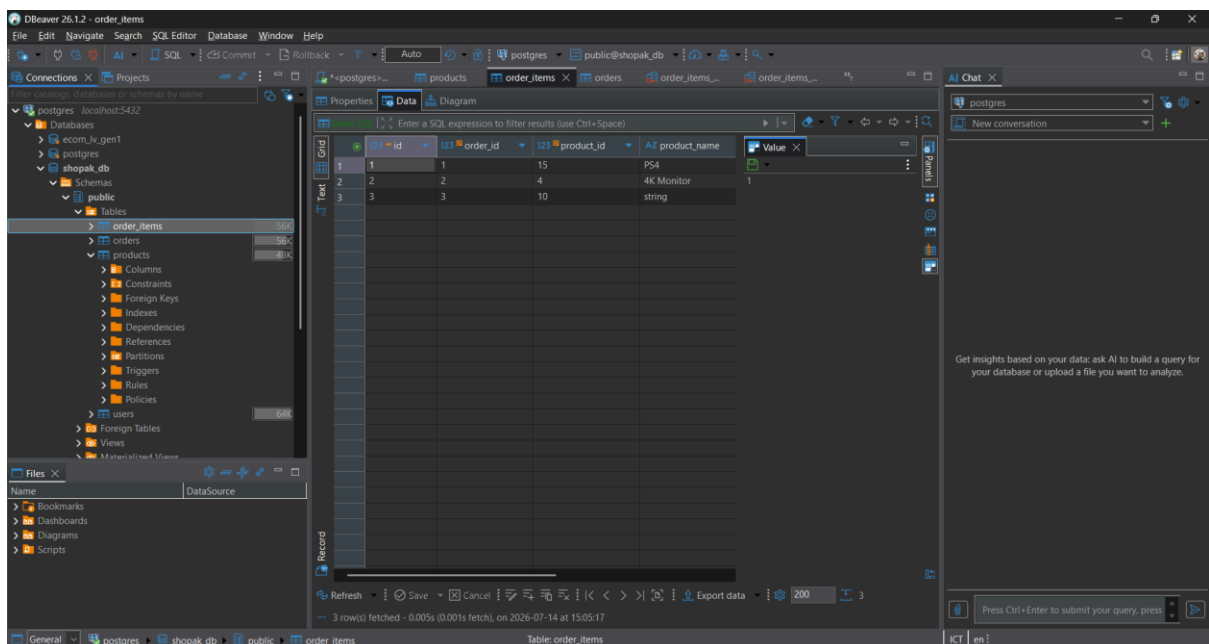
a. Tạo bảng orders và order_items với quan hệ 1-n(Orders và OrderItems).

Bảng Orders:



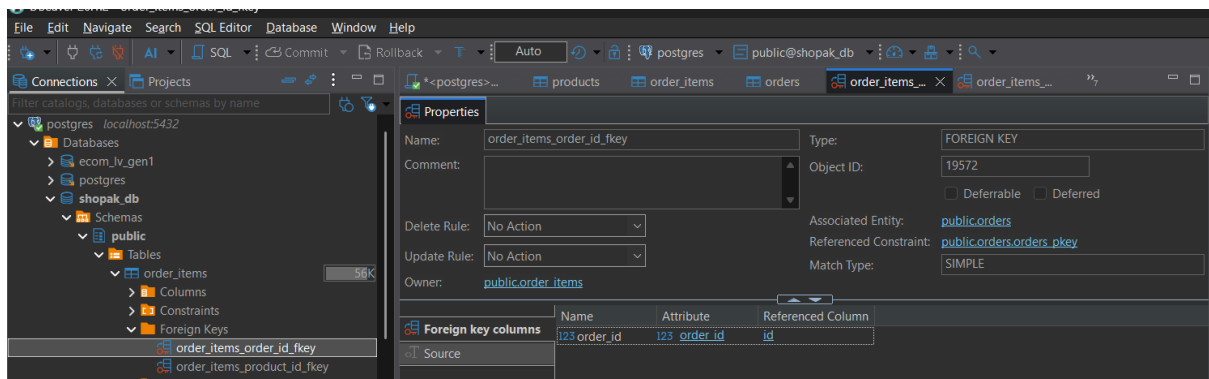
id	user_id	status	total_amount
1	4	SHIPPED	4000
2	6	COMPLETED	450
3	2	PLACED	0

Bảng orders_items:



order_id	product_id	product_name
1	15	PS4
2	4	4K Monitor
3	10	string

Bảng order_items và bảng order_id liên kết tới orders.id (quan hệ 1-n):



b. Chạy lại Base.metadata.create_all() hoặc migration để cập nhật DB.

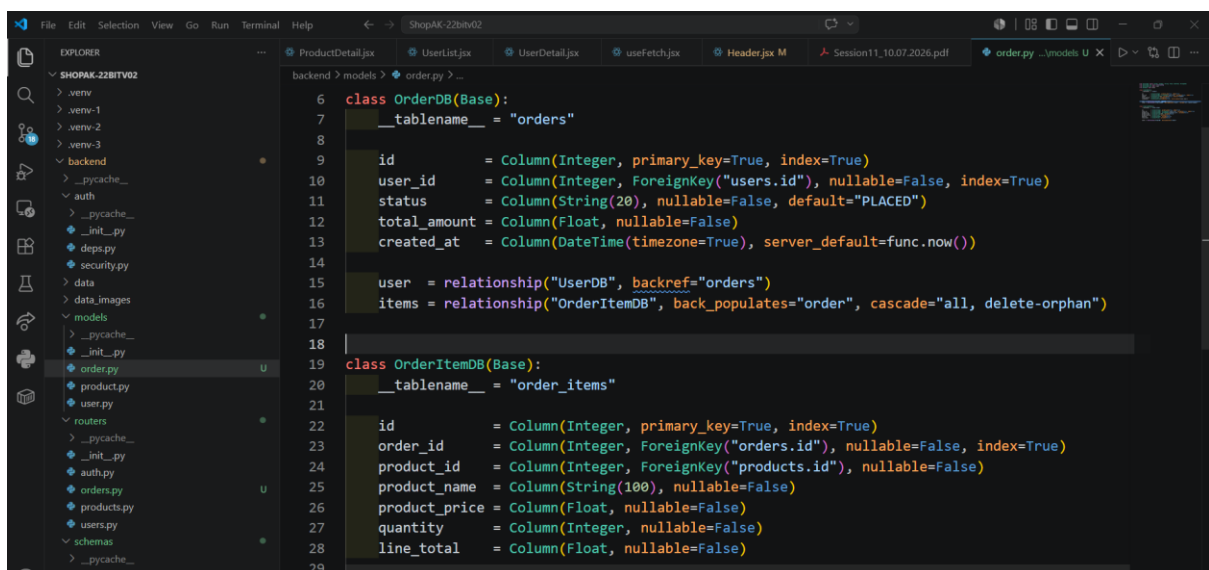
```
$ uvicorn main:app --reload
2026-07-14 15:19:24,373 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_catalog.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRAY[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspace_1)s
2026-07-14 15:19:24,373 INFO sqlalchemy.engine.Engine [cached since 0.005174s ago] {'table_name': 'orders', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspace_1': 'pg_catalog'}
2026-07-14 15:19:24,373 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_catalog.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRAY[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspace_1)s
2026-07-14 15:19:24,374 INFO sqlalchemy.engine.Engine [cached since 0.006108s ago] {'table_name': 'order_items', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspace_1': 'pg_catalog'}
2026-07-14 15:19:24,374 INFO sqlalchemy.engine.Engine COMMIT
```

Ở đây ta có thể thấy log sql CREATE TABLE orders, CREATE TABLE order_items (Base.metadata.create_all() đã chạy)

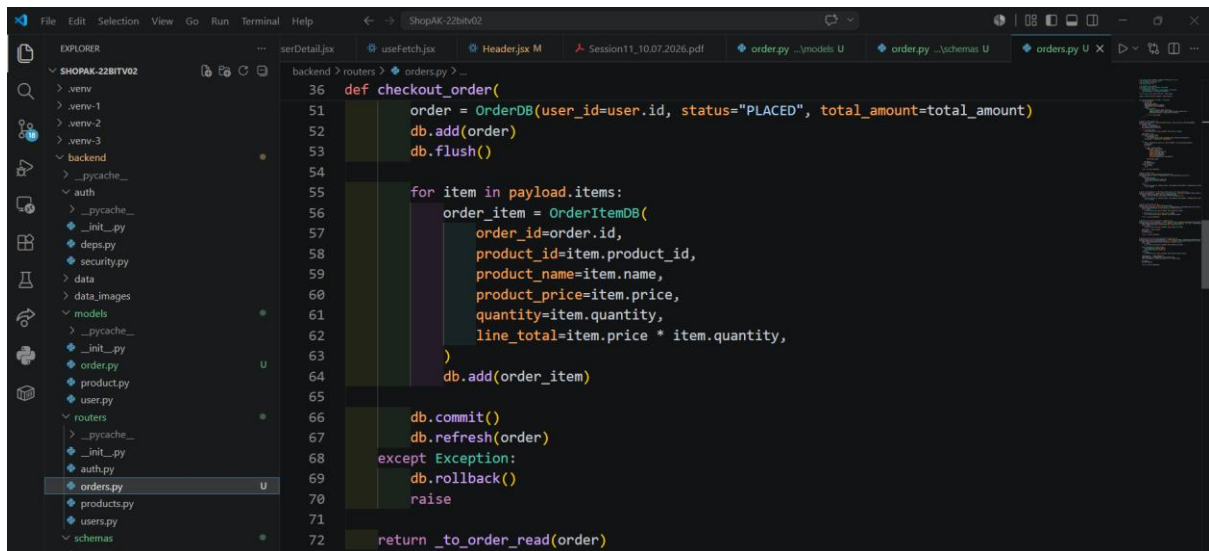
2. Backend

a. Code

2 class OrderDB, OrderItemDB với relationship



hàm checkout_order (ở dưới hình ảnh ta có thể thấy rõ transaction: db.add, db.flush, vòng lặp tạo order_items, db.commit, db.rollback trong except)

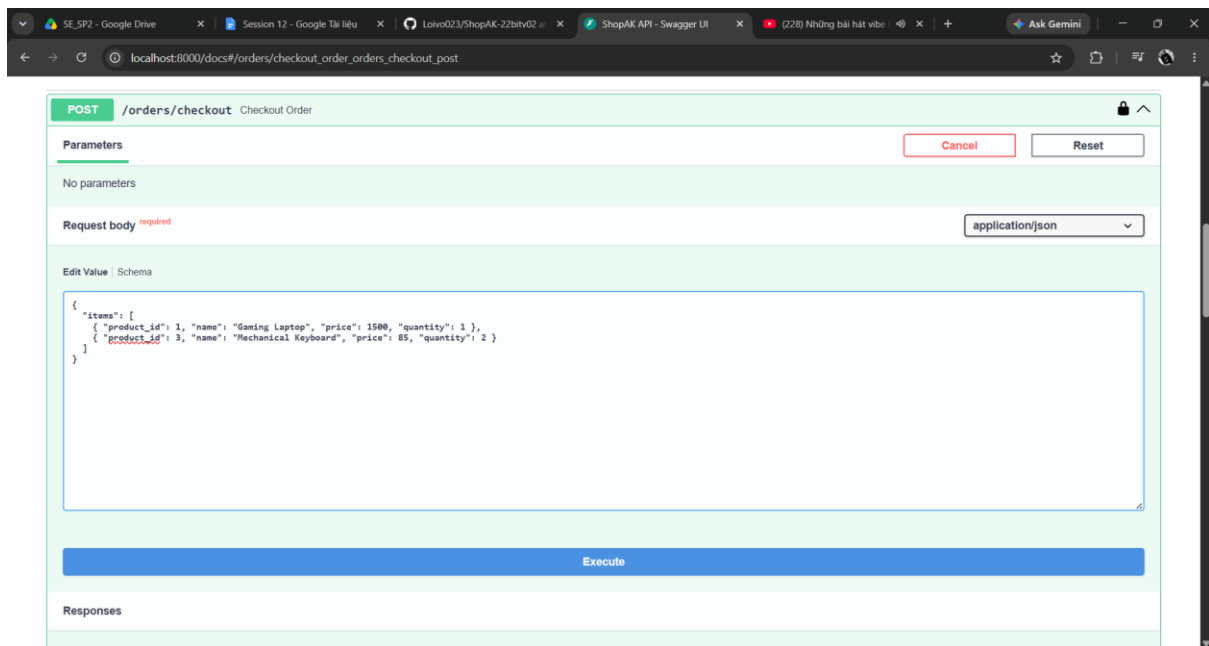


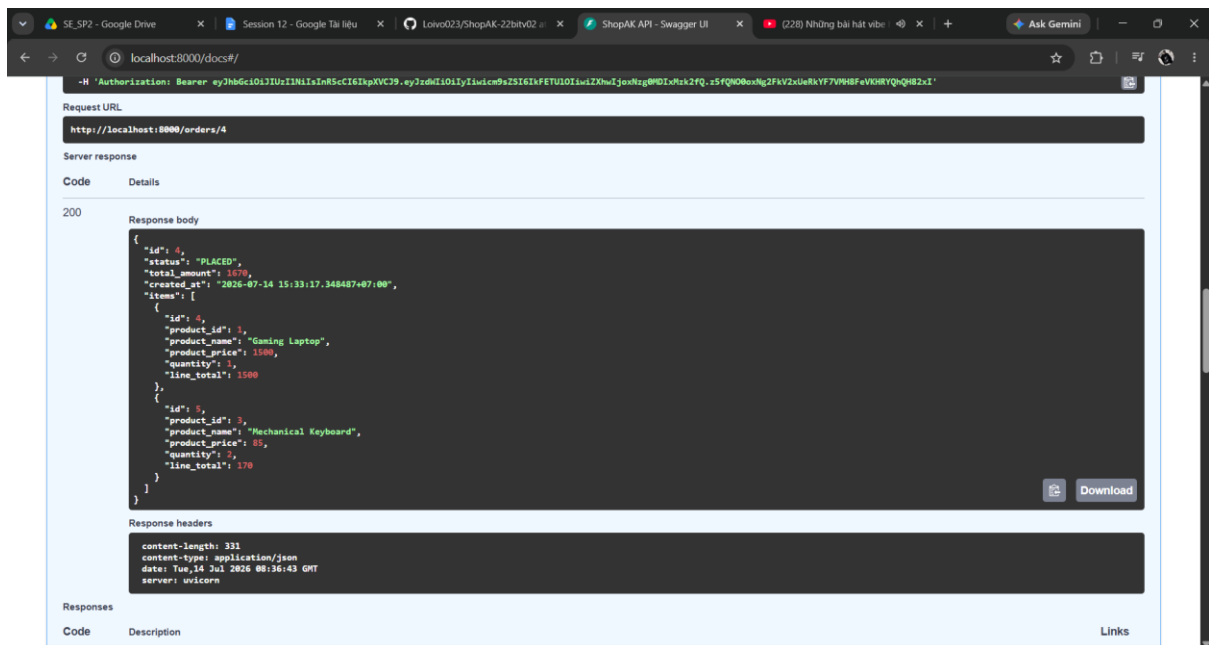
```
36 def checkout_order(  
51     order = OrderDB(user_id=user.id, status="PLACED", total_amount=total_amount)  
52     db.add(order)  
53     db.flush()  
54  
55     for item in payload.items:  
56         order_item = OrderItemDB(  
57             order_id=order.id,  
58             product_id=item.product_id,  
59             product_name=item.name,  
60             product_price=item.price,  
61             quantity=item.quantity,  
62             line_total=item.price * item.quantity,  
63         )  
64         db.add(order_item)  
65  
66         db.commit()  
67         db.refresh(order)  
68     except Exception:  
69         db.rollback()  
70         raise  
71  
72     return _to_order_read(order)
```

b. Test qua Swagger

Trước tiên Authorize bằng token của 1 user

Sau đó dùng POST /orders/checkout sau khi điền thông tin sản phẩm sau khi excute thành công sẽ trả kết quả response 201 có id, status: "PLACED", total_amount đúng ($1500 + 85 \times 2 = 1670$), và items đầy đủ





Thêm `ordersApi` với 3 hàm: `checkout`, `getMyOrders`, `getOrderByid`.

```
frontend > src > api > JS ordersApi.js > ordersApi > checkout
1 import axiosClient from "../axiosClient";
2 import { handleApiError } from "../errorHandler";
3
4 export const ordersApi = {
5   async checkout(cartItems) {
6     try {
7       const payload = {
8         items: cartItems.map((item) => ({
9           product_id: item.id,
10          name: item.name,
11          price: item.price,
12          quantity: item.quantity,
13        })),
14      };
15      const response = await axiosClient.post("/orders/checkout", payload);
16      return response.data;
17    } catch (error) {
18      handleApiError(error, "Failed to place order");
19      throw error;
20    }
21  },
22
23   async getMyOrders() {
24     try {
```

```
JS ordersApi.js U X JS authApi.js JS token.js M JS useAuth.js AdminRoute.jsx Session09_02.07.2026.pdf RegisterPage.jsx PrivateRoute
frontend > src > api > JS ordersApi.js > ordersApi > checkout
4 export const ordersApi = {
5   async checkout(cartItems) {
6     try {
7       const payload = {
8         items: cartItems.map((item) => ({
9           product_id: item.id,
10          name: item.name,
11          price: item.price,
12          quantity: item.quantity,
13        })),
14      };
15      const response = await axiosClient.post("/orders/checkout", payload);
16      return response.data;
17    } catch (error) {
18      handleApiError(error, "Failed to place order");
19      throw error;
20    }
21  },
22
23   async getMyOrders() {
24     try {
25       const response = await axiosClient.get("/orders/my");
26       return response.data;
27     } catch (error) {
28       handleApiError(error, "Failed to fetch orders");
29       throw error;
30     }
31  },
32
33   async getOrderById(id) {
34     try {
35       const response = await axiosClient.get(`/orders/${id}`);
36       return response.data;
37     } catch (error) {
38       handleApiError(error, "Failed to fetch order details");
39       throw error;
40     }
41  },
42 }
```

Trong CartPage, implement nút “Checkout”:

- Gửi cart items lên POST /orders/checkout.
- Xóa cart nếu success.
- Điều hướng đến /orders/:id.

```
3
9 const handleCheckout = async () => {
10   if (items.length === 0) return;
11   setPlacingOrder(true);
12   setError("");
13 }
```

- Tạo OrderHistoryPage để list tất cả đơn của user.

```

1 import { useEffect, useState } from "react";
2 import { Link } from "react-router-dom";
3 import { ordersApi } from "../api/ordersApi";
4
5 const statusColor = {
6   PLACED: "#1976d2",
7   PROCESSING: "#f59e0b",
8   SHIPPED: "#8e24aa",
9   COMPLETED: "#2e7d32",
10  CANCELED: "#e53935",
11 };
12
13 const OrderHistoryPage = () => {
14   const [orders, setOrders] = useState([]);
15   const [loading, setLoading] = useState(true);
16   const [error, setError] = useState("");
17
18   useEffect(() => {
19     const fetchOrders = async () => {
20       setLoading(true);
21       setError("");
22       try {
23         const data = await ordersApi.getMyOrders();
24         setOrders(data);

```

- Tạo OrderDetailPage để xem chi tiết 1 đơn.

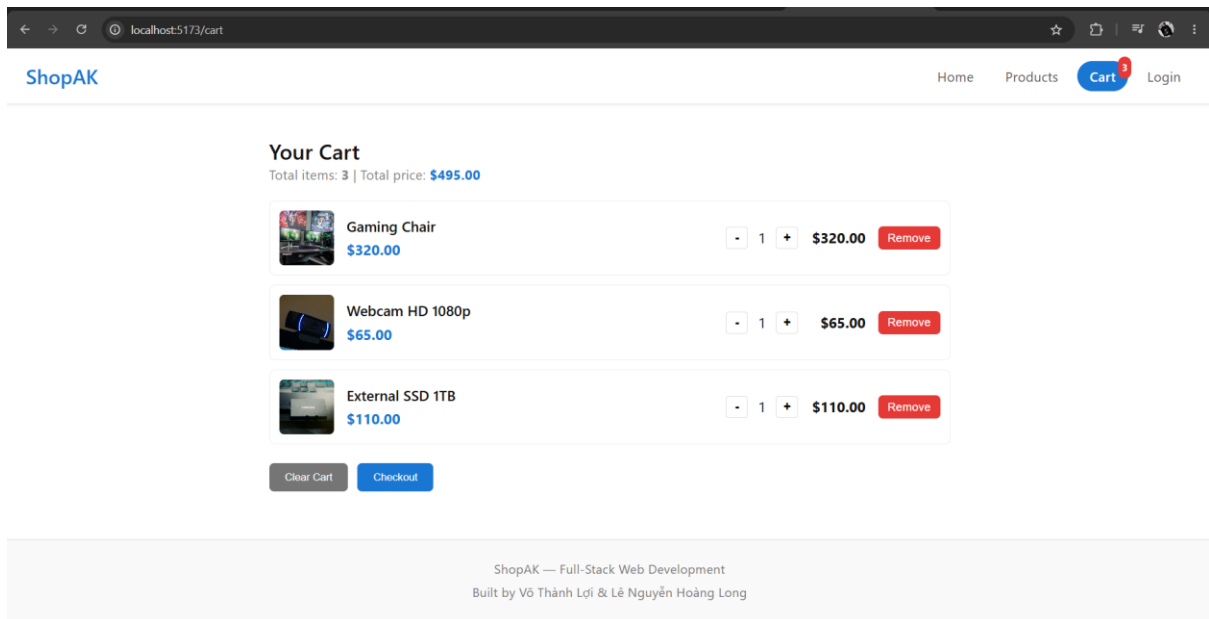
```

1 import { useEffect, useState } from "react";
2 import { useParams, Link } from "react-router-dom";
3 import { ordersApi } from "../api/ordersApi";
4 import { useAuth } from "../auth/useAuth";
5
6 const ALLOWED_STATUSES = [
7   "PLACED",
8   "PROCESSING",
9   "SHIPPED",
10  "COMPLETED",
11  "CANCELED",
12 ];
13
14 const statusColor = {
15   PLACED: "#1976d2",
16   PROCESSING: "#f59e0b",
17   SHIPPED: "#8e24aa",
18   COMPLETED: "#2e7d32",
19   CANCELED: "#e53935",
20 };
21
22 const OrderDetailPage = () => {
23   const { id } = useParams();
24   const [order, setOrder] = useState(null);

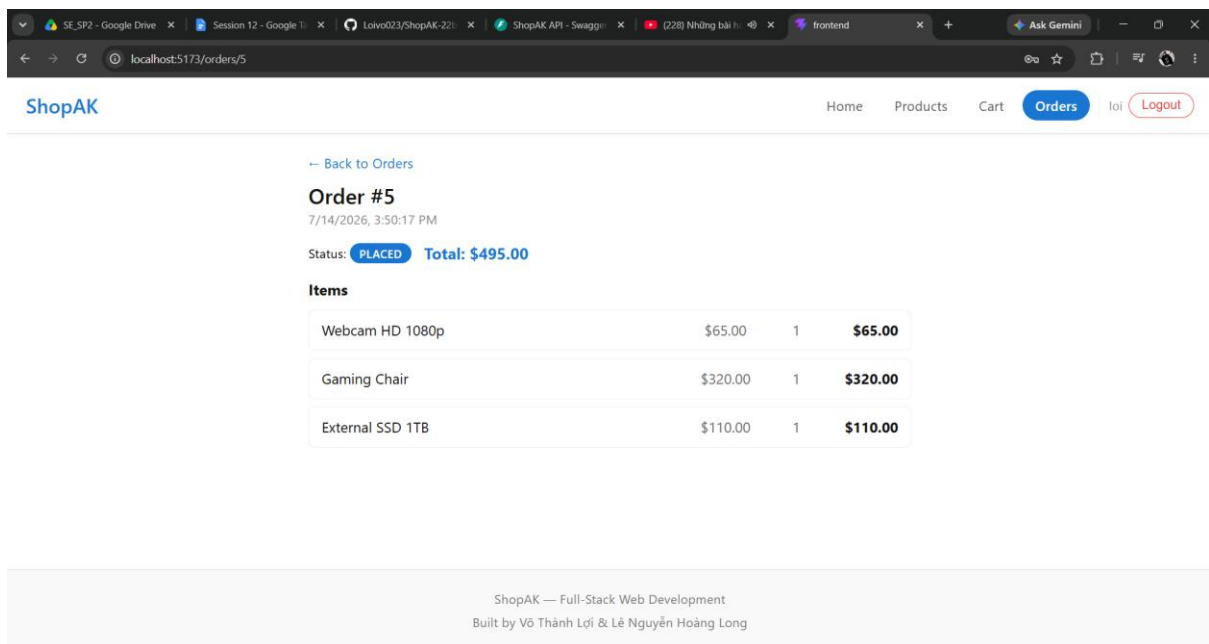
```

b. Demo

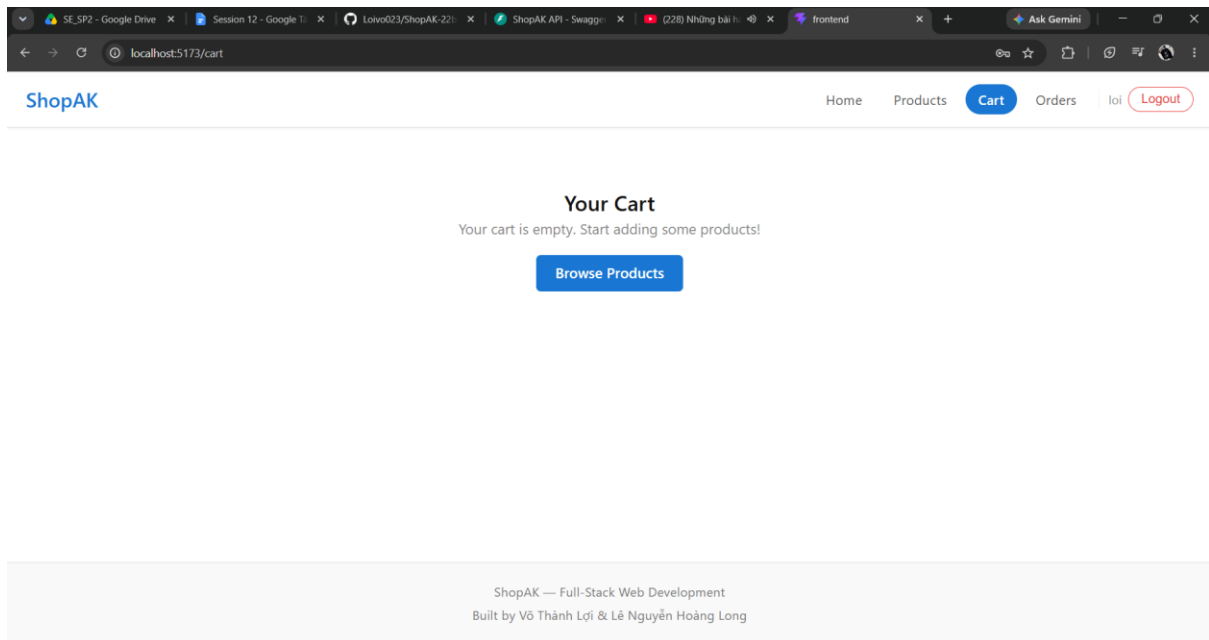
thêm 2-3 sản phẩm vào giỏ hàng



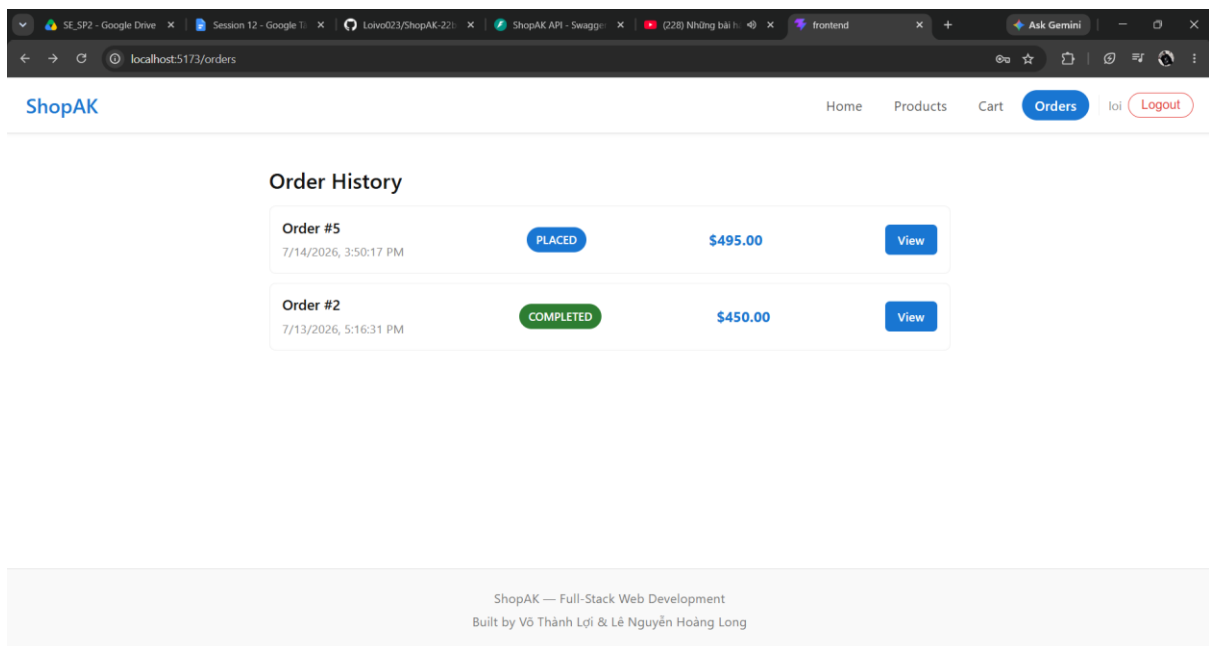
Sau đó bấm checkout sẽ loading ("Placing Order...") và đưa khách hàng vào trang lịch sử đặt hàng ("Orders") hiện chi tiết đơn vừa đặt



Vào cart lại sẽ thấy giỏ hàng đã trống do hàm clearCart() đã chạy thành công

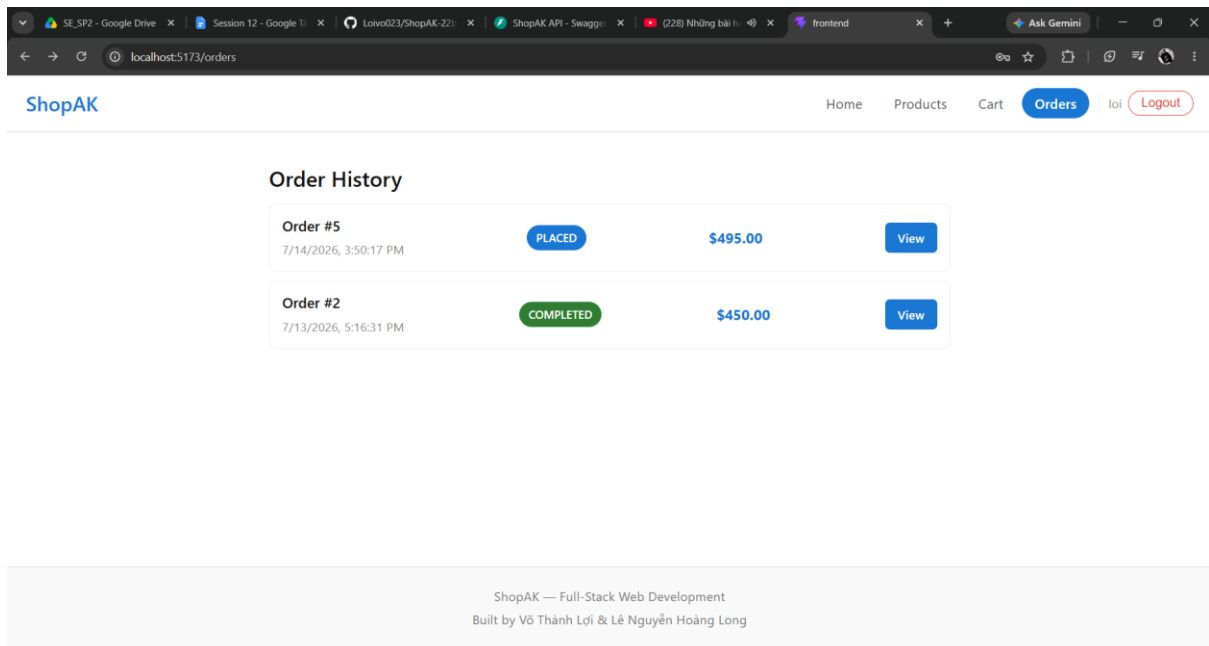


Dưới đây là danh sách lịch sử đơn hàng có đơn vừa tạo bấm vào view sẽ thấy tổng sản phẩm khách hàng đã đặt



4. Kiểm thử — User A vs User B

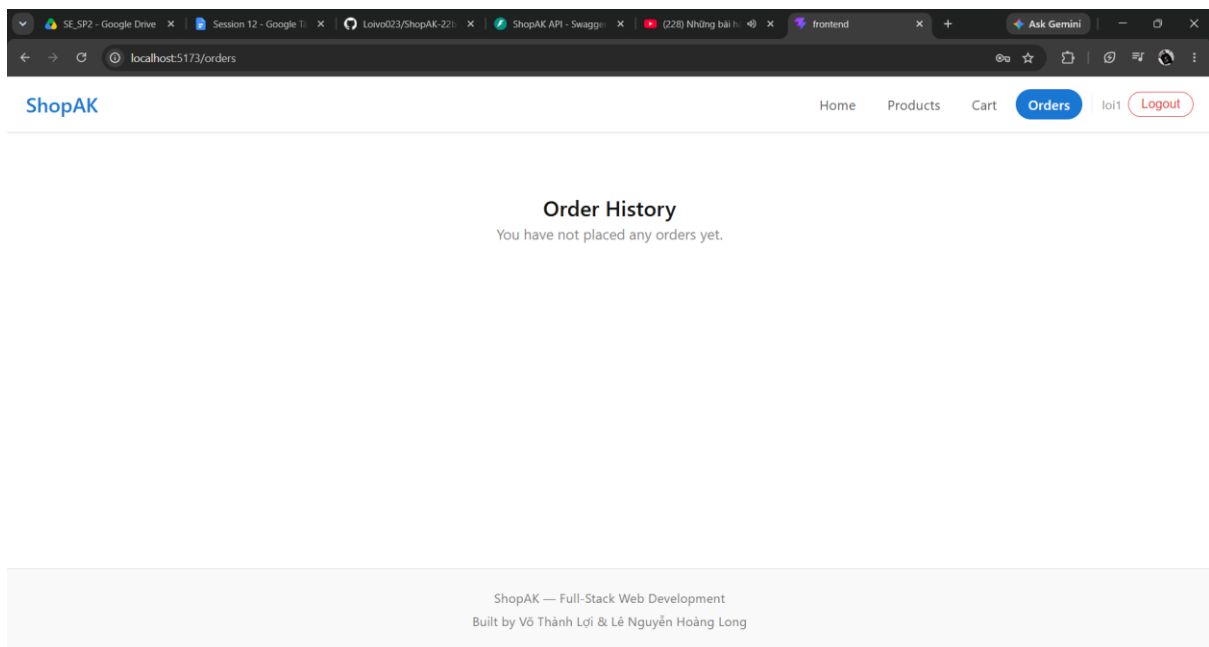
Đăng nhập **User A** role **CUSTOMER** và chúng ta checkout 1 đơn chúng ta sẽ lấy đơn có sẵn mà mình vừa đặt



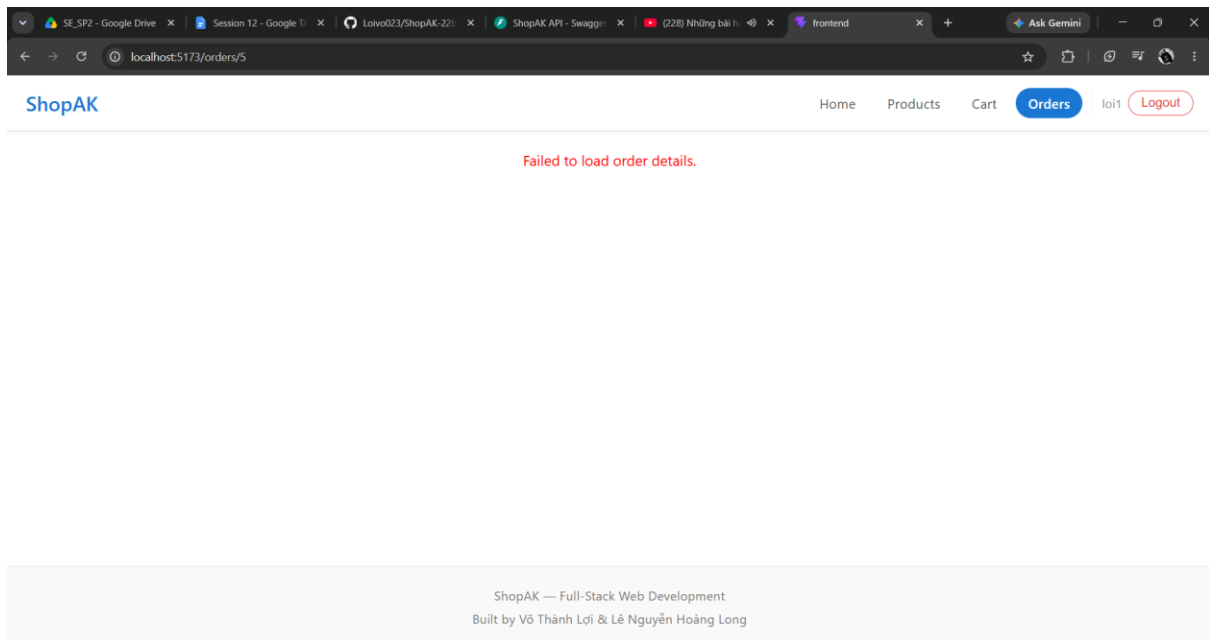
Tiếp theo đăng xuất và đăng nhập vào tài khoản khác **USER B role CUSTOMER**



Và ta có thể thấy order vẫn chưa có gì hết tài khoản mới hoàn toàn không có đơn #5 của USER A



Để cố định hơn nữa thì mình sẽ gõ URL /orders/5 của USER A trong khi đăng nhập User B ta sẽ thấy lỗi không load được đơn hàng

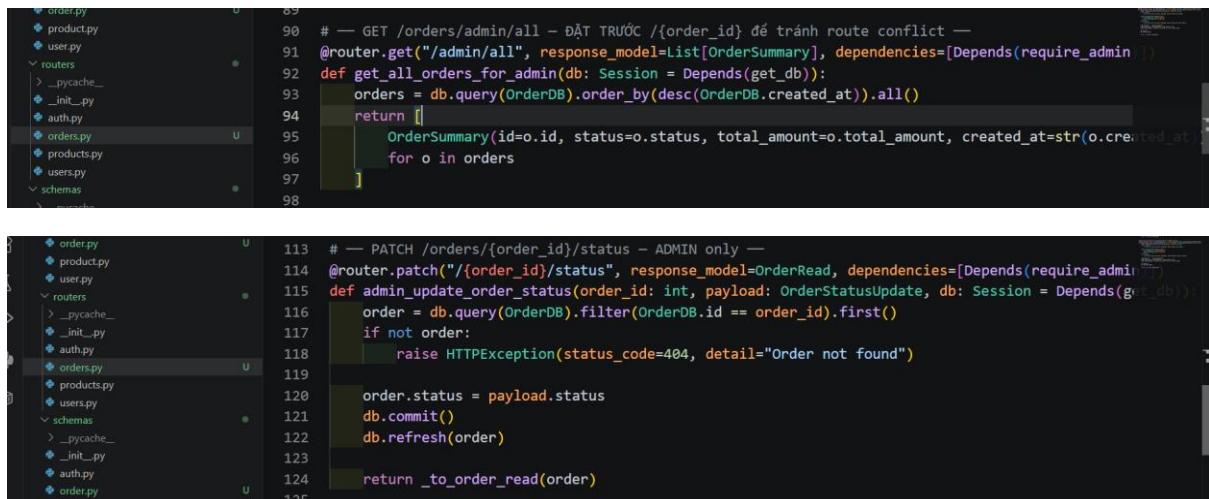


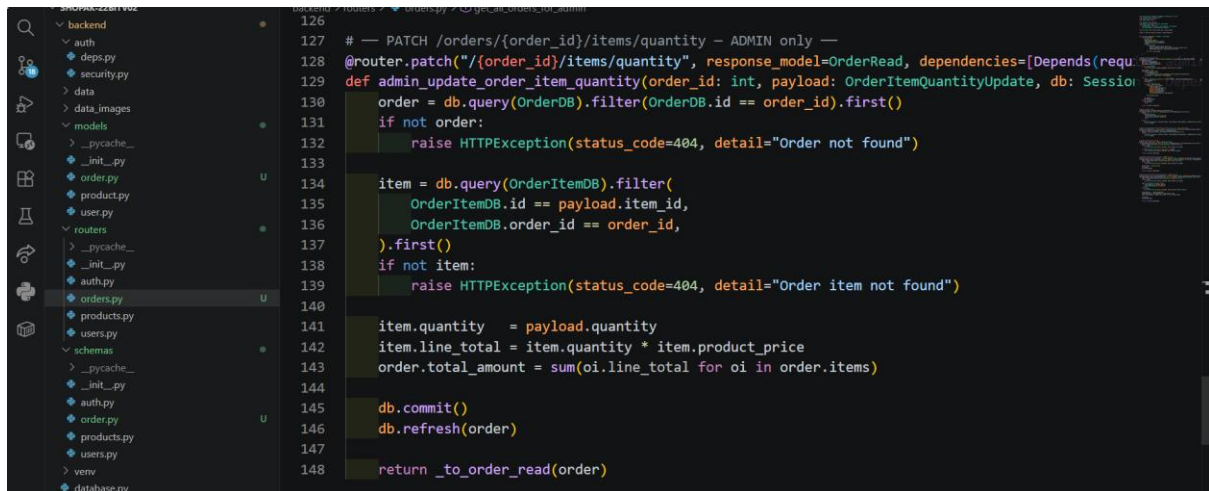
5. Admin Order Management

a. Backend code

Thêm các endpoint:

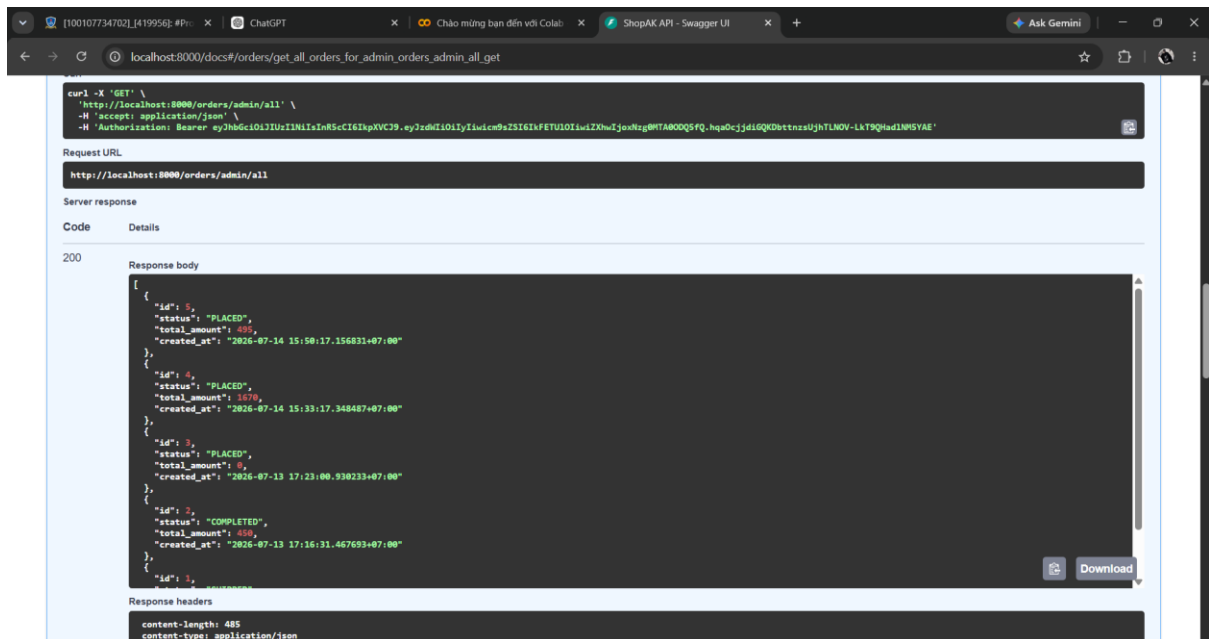
- GET /orders/admin/all (ADMIN xem tất cả đơn).
- PATCH /orders/{order_id}/status (ADMIN đổi trạng thái).
- PATCH /orders/{order_id}/items/quantity (ADMIN đổi số lượng 1 item).



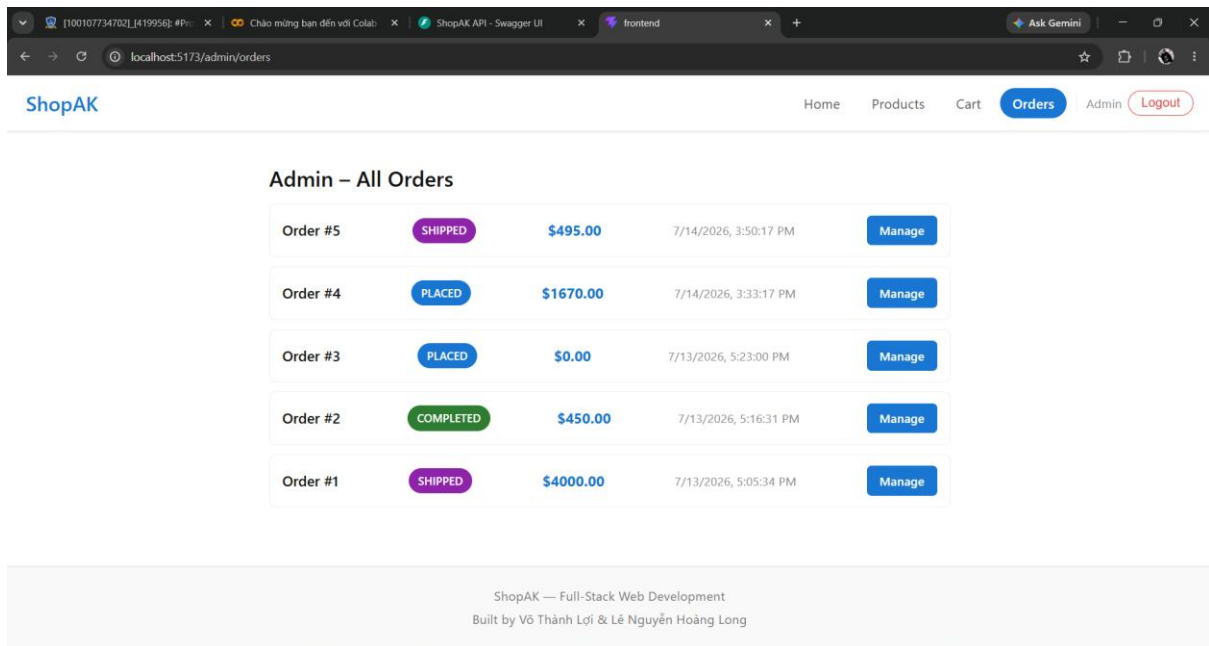


b. Test qua Swagger (Authorize bằng token ADMIN)

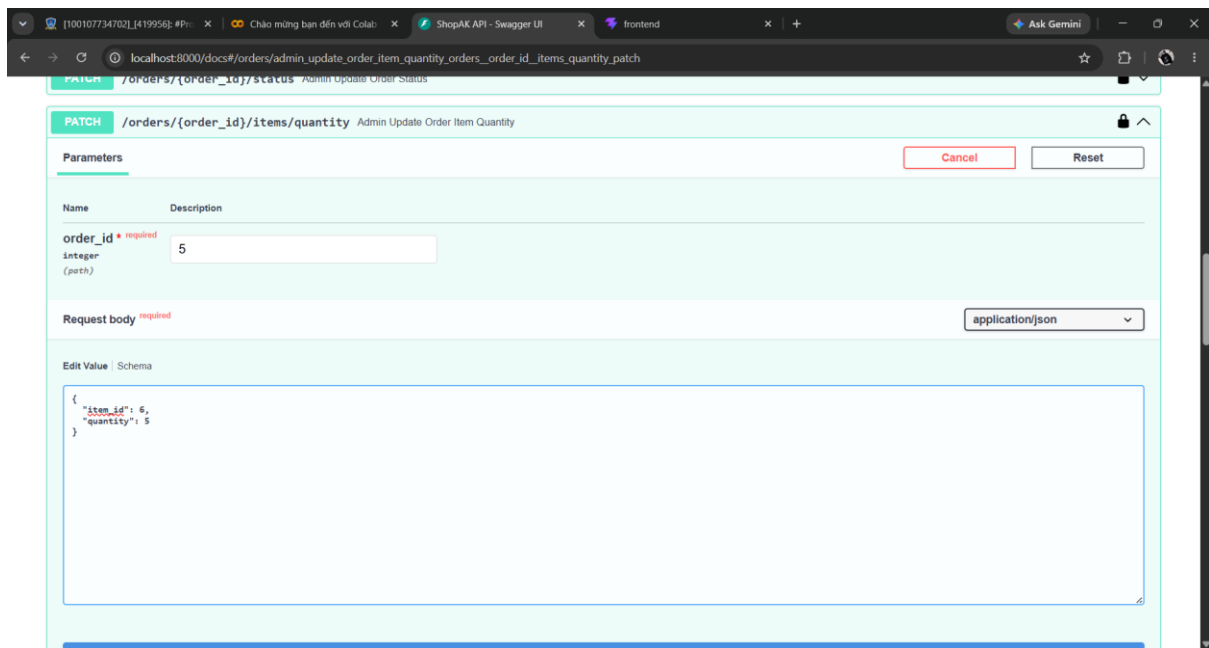
GET /orders/admin/all trả về kết quả response chứa đơn của nhiều user khác nhau (không chỉ của riêng ADMIN)

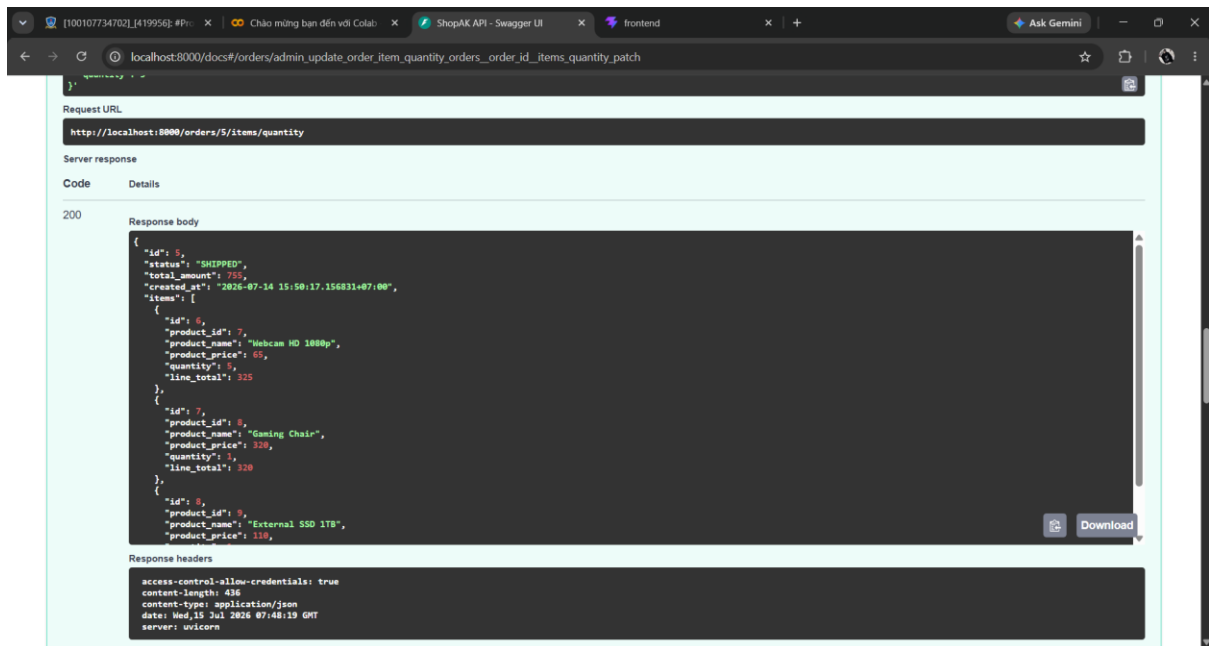


PATCH /orders/{id}/status đổi ID order thứ 5 sang thành {"status": "SHIPPED"} sau khi excute response status đã đổi sang SHIPPED

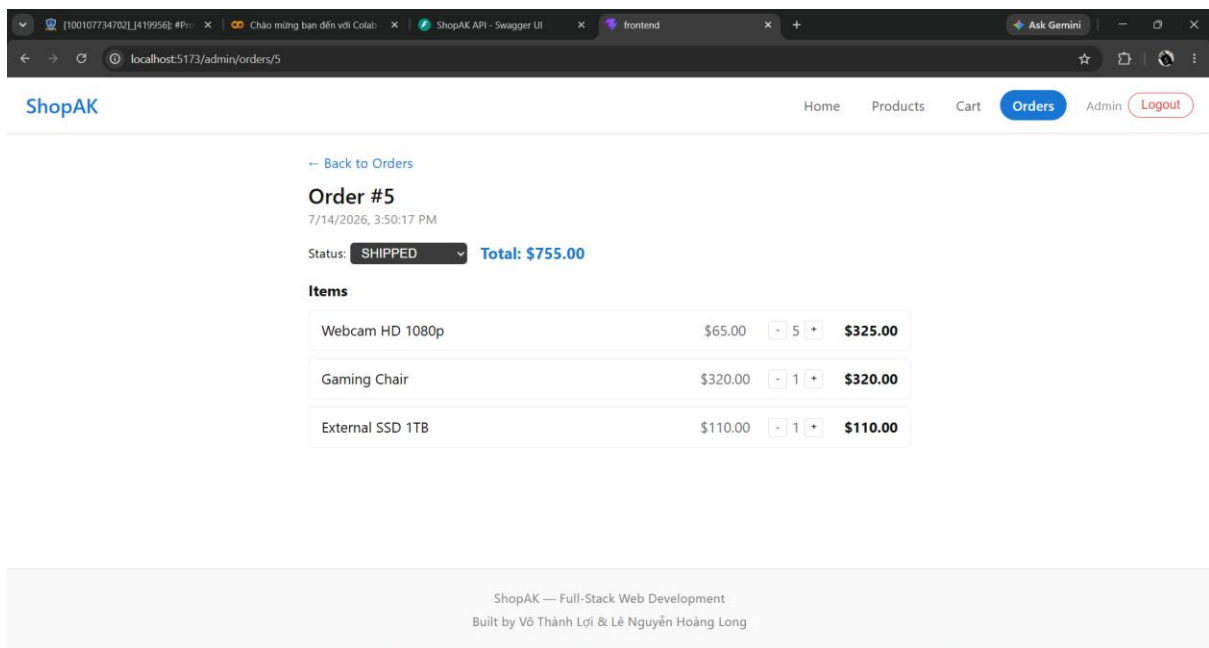


PATCH /orders/{id}/items/quantity {"item_id": 6, "quantity": 5} sau khi excute sẽ trả về response line_total và total_amount đã tự tính lại đúng



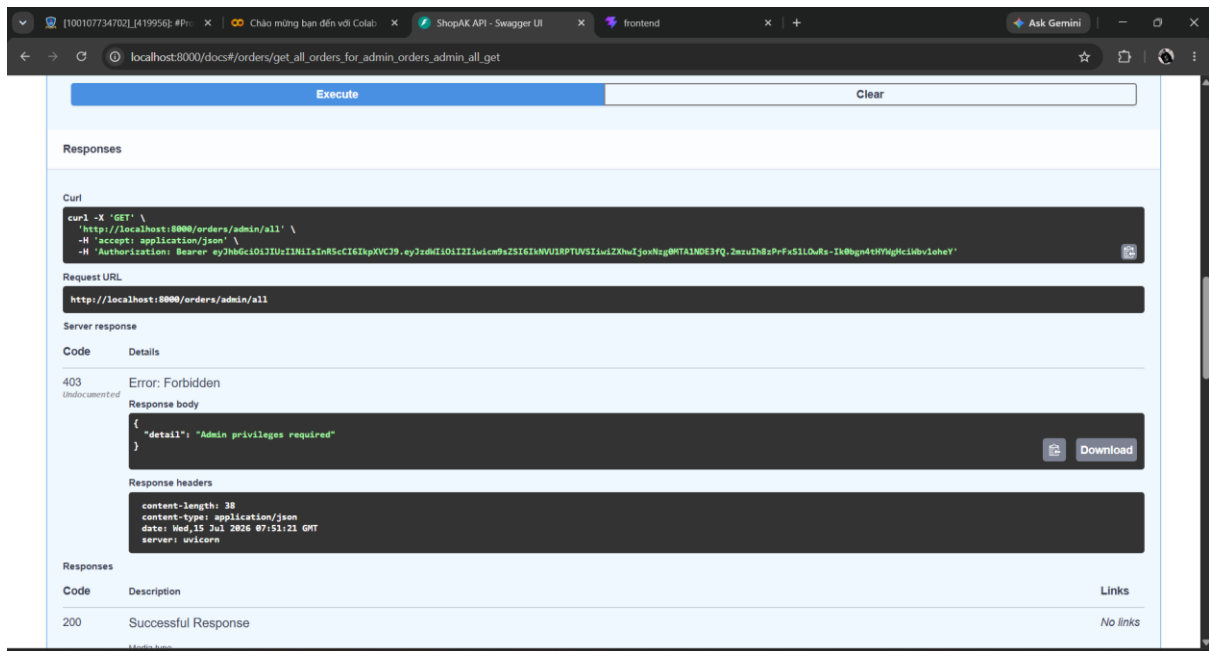


Kết quả bên web:



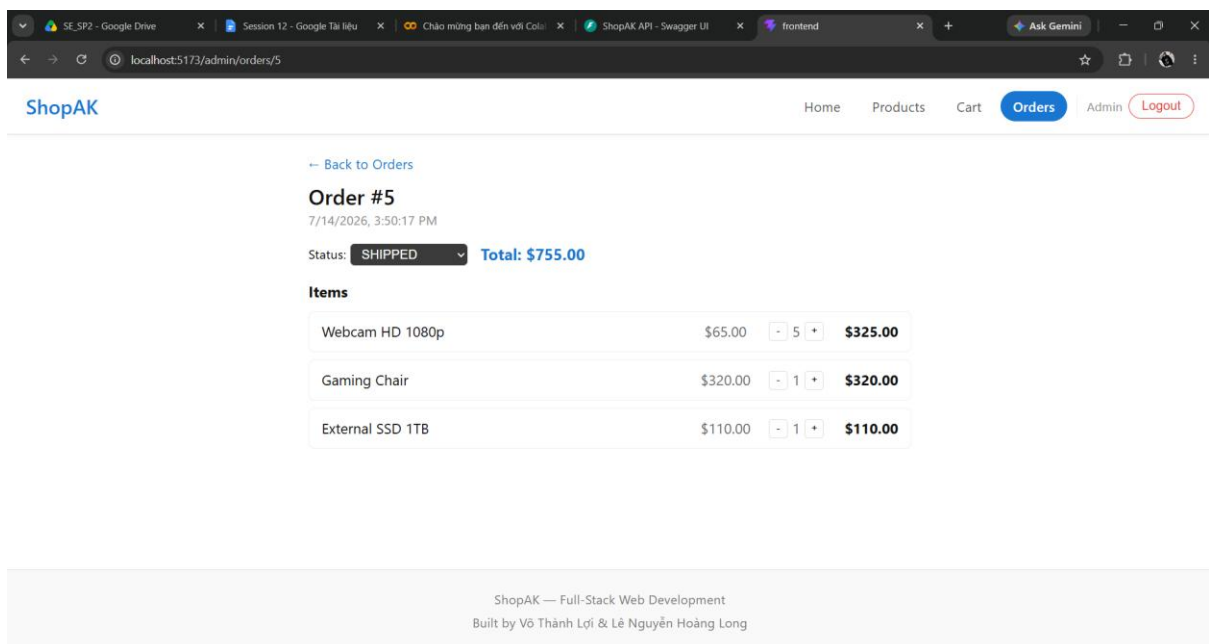
c. Test bằng token CUSTOMER (user bị chặn do không phải admin)

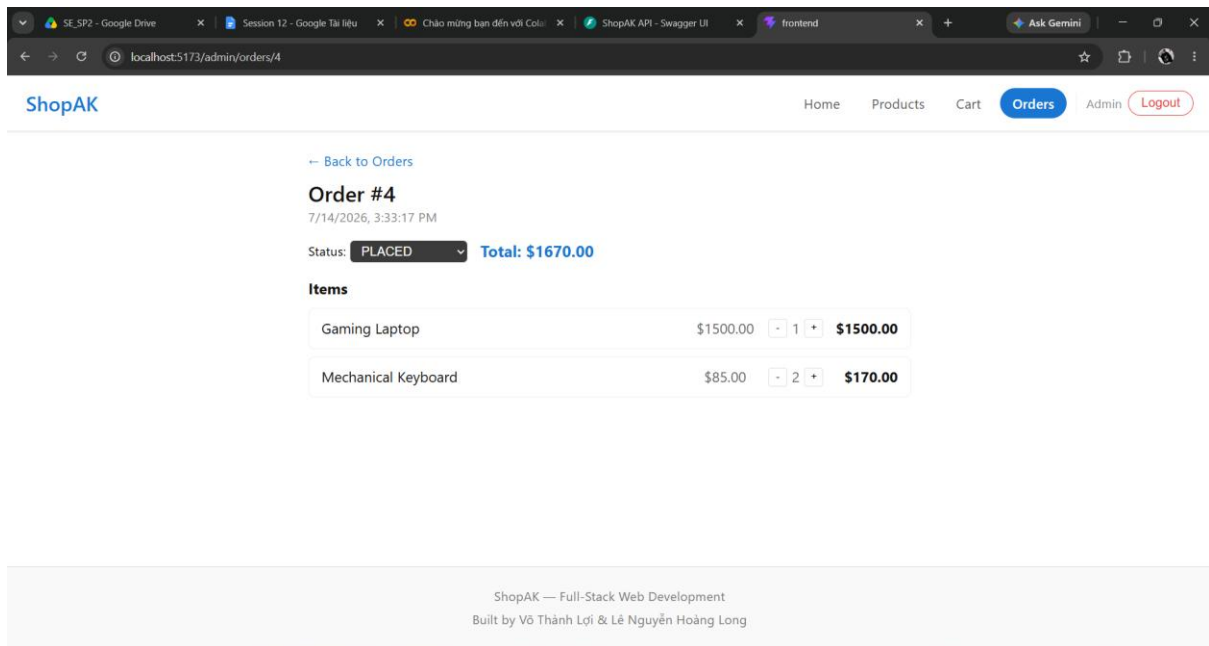
Đổi lại Authorize sang token CUSTOMER và gọi **GET /orders/admin/all** sau khi excute sẽ trả về kết quả response **403 "Admin privileges required"**



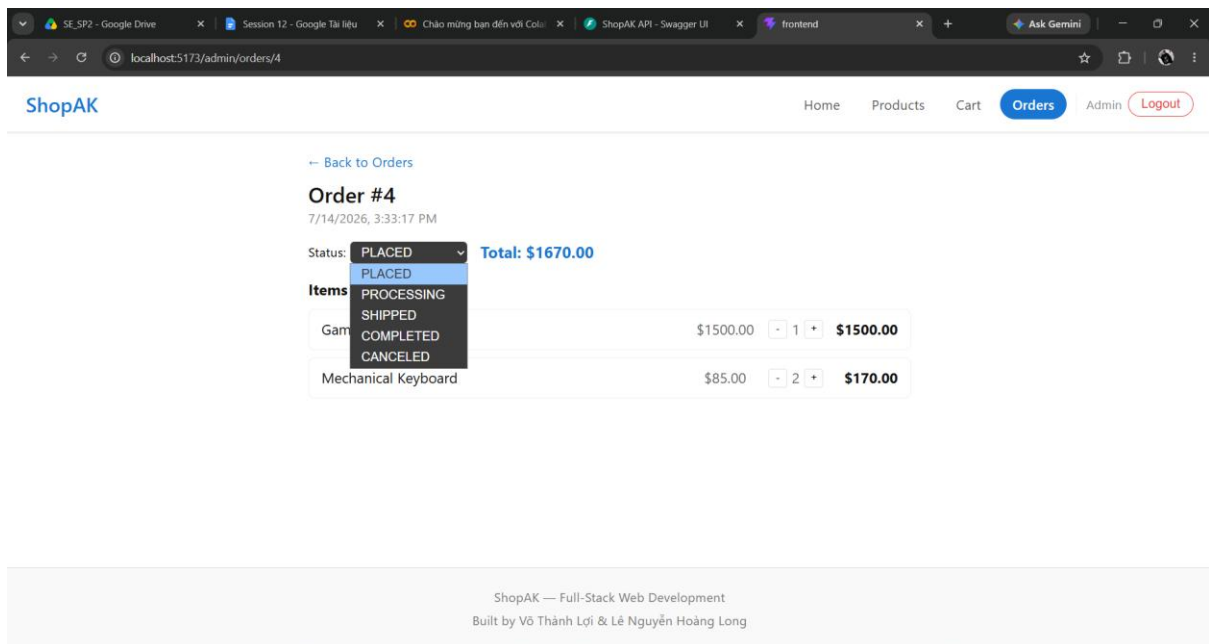
d. Frontend demo

- Đăng nhập ADMIN và vào /admin/orders check danh sách **tất cả đơn** của mọi user

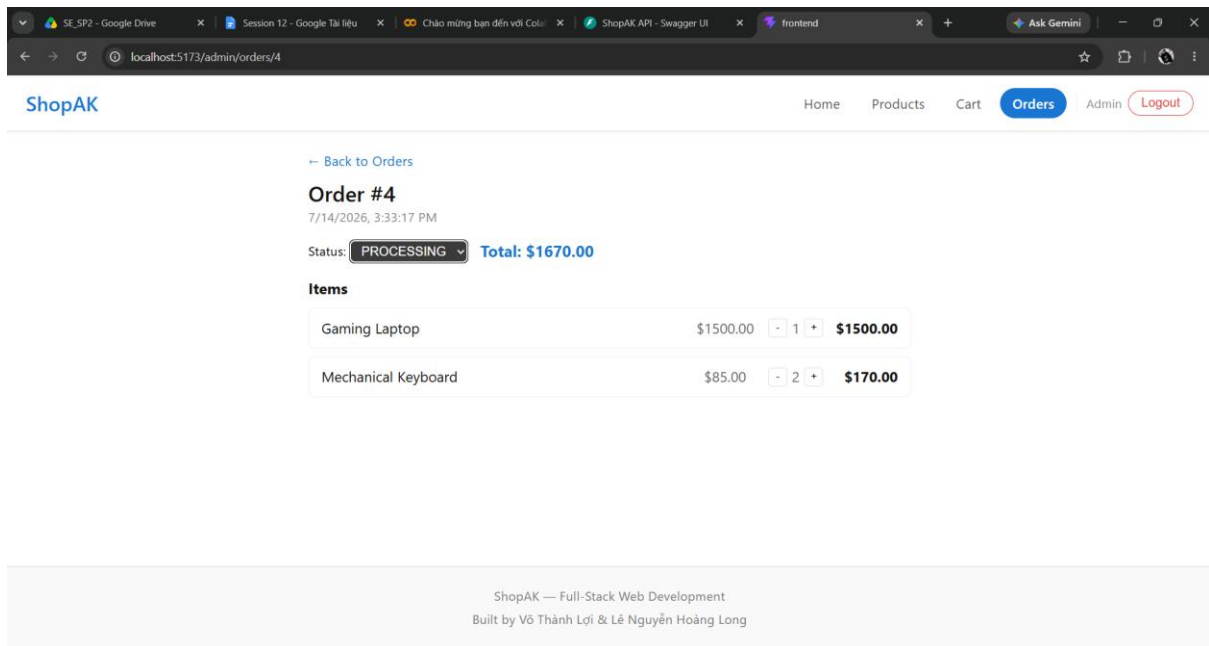




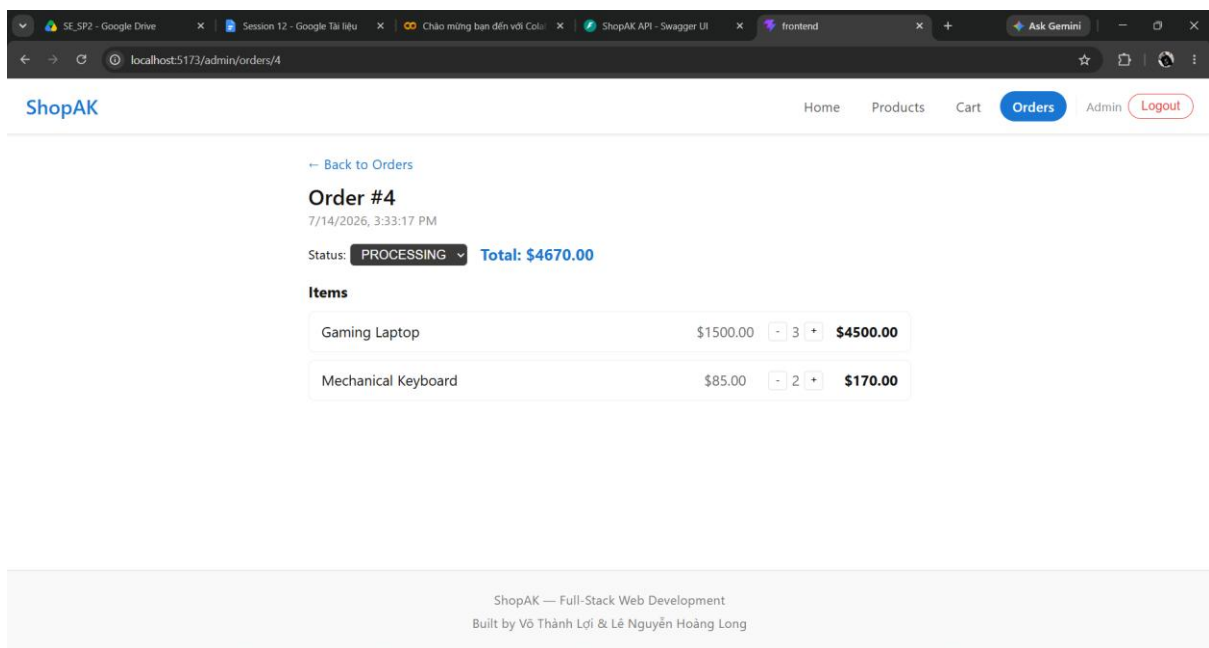
- b. Click **Manage** vào 1 đơn và OrderDetailPage hiện **dropdown đổi status** và nút **+/-** cho từng item (vì isAdmin === true)



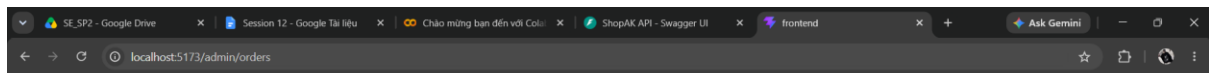
- c. Đổi status sang Processing bằng dropdown và sẽ thấy trạng thái cập nhật ngay trên UI



- d. Bấm +/- đổi quantity 1 item và chúng ta sẽ thấy line_total và total_amount tự cập nhật



- e. Đăng xuất, đăng nhập bằng **CUSTOMER** vào thẳng /admin/orders chúng ta sẽ thấy kết quả bị chặn (Access Denied)



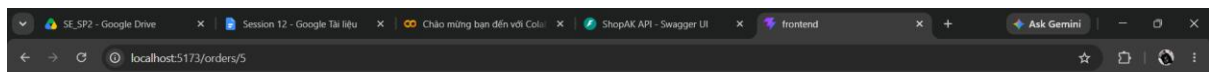
ShopAK

Home Products Cart Orders [Logout](#)

Access Denied
You do not have permission to view this page.

ShopAK — Full-Stack Web Development
Built by Võ Thành Lợi & Lê Nguyễn Hoàng Long

- f. Vào `/orders/:id` của chính CUSTOMER mà chúng ta đăng nhập sẽ thấy **chỉ hiển thị text**, không có dropdown/nút +/- (vì `isAdmin === false`)



ShopAK

Home Products Cart **Orders** [Logout](#)

[← Back to Orders](#)

Order #5

7/14/2026, 3:50:17 PM

Status: **SHIPPED** Total: **\$755.00**

Items

Webcam HD 1080p	\$65.00	5	\$325.00
Gaming Chair	\$320.00	1	\$320.00
External SSD 1TB	\$110.00	1	\$110.00

ShopAK — Full-Stack Web Development
Built by Võ Thành Lợi & Lê Nguyễn Hoàng Long